



INTERNAL

Developer Guide

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.0

Document Version	2.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Internal
Owner	Platform Engineering
Status	Released
Last Updated	2026-06-29
Document Type	Developer guide

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0 RC1; QynCore internal communication; Unified AI Workspace patterns.

Table of Contents

- 1. Repository structure
- 2. Coding standards
- 3. Service patterns
- 4. Controller / resource patterns
- 5. Migrations
- 6. Seeders
- 7. Tests
- 8. How to add a module
- 9. How to add an AI adapter
- 10. How to add a skill
- 11. How to add a compliance framework
- 12. How to add API endpoints
- 13. How to write documentation
- 14. Definition of Done
- Working on Quenyx AI (Unified AI Workspace — Sprint 20)

11 — Developer Guide

Audience: New engineers. **Goal:** Get productive in the Quenyx monorepo and ship changes that match platform conventions.

1. Repository structure

```

quenyx-saas/
├─ backend/      Laravel API (PHP 8.3) – services, QCIF engines, AI platform
│  └─ app/Http/Controllers/**      thin controllers
│  └─ app/Services/**              business logic (Compliance, Ai, QuenyxAI)
│  └─ app/Contracts/**             interfaces (e.g. AiModuleAdapterInterface)
│  └─ app/Models/**                Eloquent models
│  └─ config/**                    ai.php, compliance.php, quenyx_ai.php, ...
│  └─ database/migrations/**        65 migrations
│  └─ database/seeder/**            incl. ComplianceCorpusSeeder
│  └─ routes/**                     api.php + per-domain route files
├─ frontend/      React + Vite + TS – SPA, platformRegistry.ts
├─ gateway/        Node edge/proxy
├─ agent/          QynSight host agent
└─ docs/           documentation (this pack: docs/quenyx-v1/)

```

2. Coding standards

- **Controllers are thin:** validate → authorize → delegate to a service → return a resource/array.
- **Business logic lives in services** under `app/Services/**` ; keep them deterministic and testable.
- **UUIDs/codes** in public payloads for domain entities — never leak raw auto-increment IDs.
- **No hardcoded model names;** resolve from config/env.
- **Fail closed:** invalid input or missing entitlement → reject, don't degrade silently.
- Follow PSR-12 for PHP; the React side follows the repo's ESLint/TS config.

3. Service patterns

- One service = one responsibility (e.g. `ComplianceRetrievalService` , `RagIndexService`).
- Services receive dependencies via constructor injection (bound in `AppServiceProvider`).
- Cross-cutting AI access goes through `AiProviderRegistry` only.

4. Controller / resource patterns

- Group routes by domain in `routes/<domain>.php` , required from `routes/api.php` .
- Apply middleware at the group level: `auth:sanctum` + `project.qynshield` / `project.module:*` + the right named throttle.

- Return JSON arrays or Eloquent API Resources; keep response shapes stable (docs derive from them).

5. Migrations

- New migration per change; **never edit an applied migration** — fix forward.
- UUID primary keys for new domain entities; add indexes for query paths; preserve immutability for corpus tables.

6. Seeders

- Core data via `DatabaseSeeder` ; corpus via `ComplianceCorpusSeeder` + `compliance:seed-source-documents` .
- **No fake data** — corpus content must come from official source documents and pass the validator.

7. Tests

- `php artisan test` (feature + unit). Filters: `--filter=Compliance` , `--filter=AI` .
- Write tests for new services (deterministic inputs → deterministic outputs) and for route authorization/entitlement.

8. How to add a module

1. Add the module to the **frontend** `platformRegistry.ts` (keep it registered; sidebar visibility is controlled by the flag — don't re-enable hidden modules without product sign-off).
2. Add backend entitlement handling (module key, subscription) and `project.module:<key>` middleware if it needs gating.
3. Register the module in the backend AI catalog (`config/quenyx_ai.php`) with its AI readiness.

9. How to add an AI adapter

1. Implement `App\Contracts\QuenyxAI\AiModuleAdapterInterface` (`moduleKey` , `supportedSkills` , `supportedContexts` , `buildContext` , `buildReasoning` , `buildPrompt`).
2. Reuse existing module services — **do not duplicate business logic**.
3. Register it at boot: `QuenyxAiPlatform::registerAdapter($adapter)` in `AppServiceProvider::boot` .
4. Its `module_catalog` status flips to `production` ; the capability catalog updates automatically.

10. How to add a skill

1. Implement the skill class under `app/Services/Ai/Skills/**` ; it must **reuse a deterministic service** and make **no AI call**.

2. Register it in `config/ai.php` → `skills.registered` with a key, priority, and `enabled` flag.

11. How to add a compliance framework

1. Prepare official **source documents** + a manifest under `database/corpus/<authority>/<framework>/`.
2. Seed via `ComplianceCorpusSeeder` and `compliance:seed-source-documents --framework=... --release=...`.
3. The validator enforces no fake data, code uniqueness, and provenance. Create an active **revision**.

12. How to add API endpoints

1. Add the route to the relevant `routes/<domain>.php` under the correct middleware group.
2. Implement the controller method (thin) + the service.
3. Run `php artisan route:list` to confirm registration and **no collisions**.
4. Update Doc 08 (API Reference).

13. How to write documentation

- Update the affected doc(s) in `docs/quenyx-v1/` in the **same PR** as the code.
- Respect the status legend ( /  /  / ). **No fabrication**.
- Re-derive Doc 08 (routes) and Doc 09 (migrations) when those change.

14. Definition of Done

- ☐ Code matches service/controller conventions; no hardcoded models; UUIDs in payloads.
- ☐ Routes registered, gated, throttled; `route:list` clean.
- ☐ Migrations are new (not edited); apply cleanly.
- ☐ Tests added/updated and passing (`php artisan test`).
- ☐ AI stays off-by-default; no direct provider calls outside provider classes.
- ☐ Docs updated; status badges correct.
- ☐ No fake/sample data; corpus validator passes.

Working on Quenyx AI (Unified AI Workspace — Sprint 20)

RC1.1: the UI label is **Quenyx AI**. The canonical SPA base stays `/ai-workspace/*` (a `/quenyx-ai/*` alias in `App.tsx` redirects to it). The provider catalog is declared in `App\Services\Ai\AiProviderCatalog`; `AiProviderRegistry` decides which entries are executable (have an adapter) and which are platform-configured. Keep new catalog entries non-executable until a real adapter is added — never fabricate connectivity.

- **Backend** lives in `App\Http\Controllers\Ai\Workspace`, `App\Services\Ai\Workspace`, `App\Models\Ai\*`, `app/Http/Resources/Ai/*`, and `routes/ai-workspace.php`. Resolve + authorize a workspace through `AiWorkspaceBaseController::workspace()` (which calls `AiWorkspaceContextResolver` and `ProjectPolicy::accessAi|administerAi`), then gate fine-grained actions with `requireCapability( ... )`. Return data via the `{ success, data }` envelope (`ok()`); expose **UUIDs only**.
- **Reuse, don't duplicate:** conversations via `AiConversationRepository`; execution via `AiProviderRegistry` + `CompliancePromptOrchestrator`; skills/capabilities via `QuenyxAiPlatform`. New AI logic does not belong here.
- **Secrets:** store on `AiProviderSetting.settings` (the `encrypted:array` cast) and never return raw values — only `secret_configured`. Audit via `AiWorkspaceAuditLogger` (it strips secrets).
- **Frontend:** add a tab in `layouts/AiWorkspaceLayout.tsx`, a lazy route in `App.tsx`, a page under `pages/ai/*` using `useAiResource` + `AiView` (handles no-workspace/loading/error/empty), a method on `services/aiWorkspaceService.ts`, types in `types/aiWorkspace.ts`, and matching keys in **both** the `en` and `ar` blocks of `i18n/translations.ts`.
- **Validate:** `php artisan route:list | grep ai`, `php artisan migrate --force`, `php artisan test`, `npm run build` (+ `npm run lint`).